

Implementasi Konsep Hashing dalam Proyek

Kelompok 4:

- Christover Angelo Lasut (2306220343)
 - Matthew Immanuel Sitorus (2306221024)
 - Syahmi Hamdani (2306220532)
-

Hashing adalah sebuah proses mengubah input berupa data dengan ukuran tak terhingga menjadi output berupa karakter atau string dengan ukuran panjang yang tetap. Output inilah yang disebut sebagai hash value. Fungsi hashing memiliki karakteristik unik: bersifat satu arah sehingga hash tidak bisa dikembalikan ke output aslinya secara terbalik, sangat cepat untuk dikomputasi, dan perubahan sekecil apa pun pada data asli, bahkan satu huruf sekalipun akan menghasilkan output hash yang sama sekali berbeda. Pada pengembangan ekosistem Web3 dan Blockchain seperti proyek RWA Tokenization DApp, algoritma hashing menjadi fondasi utama untuk memastikan integritas, keamanan, dan efisiensi data.

Implementasi hashing yang paling menonjol di dalam development environment proyek kami berbasis Ethereum adalah algoritma kriptografi Keccak-256. Salah satu kasus penggunaan nyatanya ada di dalam sistem modul Governance, di PropertyDAO.sol. Dalam ini, ketika investor ingin menjalankan sebuah proposal kebijakan, aksi penjadwalan dan pengeksekusian tidak memerlukan penjabaran keseluruhan teks deskripsi proposal secara lengkap, melainkan hanya memerlukan descriptionHash (sebuah data 32-byte hash). Hal ini menghemat biaya gas secara drastis saat interaksi On-Chain, sambil tetap dapat membuktikan bahwa deskripsi instruksi proposal tersebut valid dan tidak dimanipulasi oleh pihak mana pun.

Di Ethereum Virtual Machine (EVM), hashing dimanfaatkan tanpa disadari dalam tipe data struktur state Smart Contract seperti mapping (contohnya pemetaan alamat metamask pengguna ke jumlah balance RWA tokens miliknya). EVM menggunakan teknik kalkulasi ruang memori berbasis keccak256 untuk melacak slot indeks storage secara efisien $O(1)$.

Contoh Konsep Hashing Sederhana Terkait Proyek

Sebagai demonstrasi teknis untuk membuktikan sifat dinamis dari hashing, berikut adalah contoh snippet untuk mensimulasikan perhitungan hash pada sebuah deskripsi proposal kebijakan DAO, yang dapat direplikasi melalui bantuan library web3 seperti Ethers.js yang dipakai dalam lingkungan React Frontend proyek kami:

```
// Contoh konsep Hashing menggunakan Web3 Library (Ethers.js)
const { ethers } = require("ethers");
// Data Awal
const deskripsiProposal = "Proposal Perbaikan Atap Gedung Senilai
0.5 ETH";
// Mengkalkulasikan Hash Keccak-256
const hashProposal = ethers.id(deskripsiProposal);
console.log("Hash Asli:", hashProposal);
// Output Hash akan selalu mutlak: 0x6e2f1b...abcd
// Modifikasi satu huruf kapital menjadi huruf kecil
const deskripsiManipulasi = "Proposal Perbaikan atap Gedung
Senilai 0.5 ETH";
const hashManipulasi = ethers.id(deskripsiManipulasi);
console.log("Hash Manipulasi:", hashManipulasi);
// Output Hash berubah total secara drastis dari hash asli
```

```
Teks : Proposal Perbaikan Atap Gedung Senilai 0.5 ETH
Hash : 0x98da834a8f9bbea9af052887e4de4ee76fb7245905dd6a68ce13f2e3349ef702

-----

Teks : Proposal Perbaikan atap Gedung Senilai 0.5 ETH
Hash : 0x99574402e5ac6dd65c5d0f815814b1feb685eae2043b48bb5cb76afd13ee84c6
```

Dari contoh program di atas yang bisa dijalankan di ekosistem frontend yang dikembangkan, dapat dibuktikan bagaimana modifikasi pada satu huruf 'A' ke 'a' sekalipun langsung mengganti komposisi hash value yang diproduksi. Mekanisme ini lah yang memastikan implementasi sistem tokenisasi berjalan secara aman tanpa kepercayaan tunggal (trustless), karena semua rekursi dan keabsahan dokumen dibentengi komputasi dan verifikasi hashing.